NextCaseHQ — Administrator Manual

This is the operational reference for whoever deploys, configures, and keeps NextCaseHQ running. It is written for a technical operator — someone comfortable with a Linux shell, PostgreSQL, and environment variables — not necessarily someone who works in the codebase day to day.

Every claim below is grounded in the actual repository: `db/schema.sql`, `scripts/db/migrate.js`, `package.json` / `apps/web/package.json`, `.env.example`, `apps/web/instrumentation.ts`, the library modules under `apps/web/src/lib/`, the two health-check routes, and the CI workflows under `.github/workflows/`. Where the product is honestly incomplete (a provider abstraction with no route wired to it yet, a health check that doesn't actually probe anything), this manual says so rather than describing aspirational behavior.

---

1. Overview

NextCaseHQ is a litigation and matter-management platform for advocates and law firms, built as a **pnpm/Turborepo monorepo**:

- **`apps/web`** — the real, deployable product: a Next.js 16 application (App Router, React 19) that is both the UI and the API layer (Next.js Route Handlers under `apps/web/src/app/api/*`). This is the only app you deploy to serve traffic.
- **`apps/workers`** — a minimal TypeScript scaffold (`tsc` build, no runtime dependencies of consequence). It is not a running background-job service today; nothing in `apps/web` currently depends on it at runtime.
- **`packages/*`** — shared workspace packages: `country-packs` and `ndl`, both consumed by `apps/web` (`@nextcase/country-packs`, `@nextcase/ndl`). A number of unused scaffold packages (`ai-kernel`, `ai-registry`, `crypto`, `event-bus`, `legal-kernel`, `messaging`, `observability`, `prompt-library`, `qa`, `search-engine`, `workflow-engine`, `design-system-ndl`) were removed from the workspace as part of release cleanup — none were ever imported by the running app.

The system's real infrastructure dependencies are:

- **PostgreSQL**, with the **pgvector** extension enabled, as the single system of record. All application tables enforce **Row-Level Security (RLS)** for tenant isolation — this is a real, enforced security boundary, not a convention (see §4).
- **Redis**, used to make login/admin-login rate limiting consistent across multiple running instances, and as a cache for AI context. It is optional in the sense that the app degrades to an in-memory, per-instance implementation when `REDIS_URL` is unset — but that degrade mode is explicitly called out as unsafe for a horizontally-scaled production deployment (see §3 and §7).
- **S3-compatible object storage** (real AWS S3, MinIO, Cloudflare R2, Supabase Storage's S3-compatible endpoint, DigitalOcean Spaces, or `s3rver` for local dev/test) for document file bytes. Uploads and downloads have no functioning fallback when this is unconfigured — the relevant routes fail with a clear, defined error rather than silently no-op'ing.

NextCaseHQ is multi-tenant: every domain table carries a `tenant_id` column, and PostgreSQL RLS policies — not application code — are the mechanism that prevents one tenant's data from ever being readable by another. The application connects to the database as a deliberately unprivileged role (`nextcase_app`) that cannot bypass this boundary even if application code had a bug.

---

2. Deployment

2.1 Prerequisites

Before the app can serve real traffic, provision:

1. **PostgreSQL 14+** with the `vector` extension installable (the `pgvector/pgvector` Docker image is what CI uses; a managed provider like Supabase already has it available). `pg_trgm` is also required (used for Universal Search's structured-field ranking) — it ships with stock PostgreSQL, no extra package needed.
2. **Redis** (any standard instance — self-hosted, ElastiCache, Upstash, etc.) reachable from every running instance of the app.
3. **An S3-API-compatible object storage bucket** with credentials.
4. Real secrets for every value in §3.4 (Security/Session) — do not carry the local-dev placeholders into any shared or production environment.

2.2 Build and start (real commands)

From the monorepo root, using pnpm (the project pins `packageManager: pnpm@10.30.3` in `package.json`):

```
pnpm install --frozen-lockfile
pnpm build          # runs `turbo build` across all workspace packages/apps
pnpm --filter web start   # or: cd apps/web && pnpm start (runs `next start`)
```

The root `package.json` scripts, verbatim:

Script	Command	Purpose
`build`	`turbo build`	Builds every workspace package/app (Turborepo orchestrates dependency order; caches `.next/**` and `dist/**`).
`dev`	`turbo dev`	Local development, all workspaces.
`lint`	`turbo lint`	Lints every workspace. Note: `apps/web` 's own lint script currently just echoes "No ESLint configuration found. Skipping linting." — lint is not a real gate for the web app today.
`typecheck`	`node scripts/typescript_validation.js`	Runs `tsc -p <tsconfig> --noEmit` against each mapped project (`apps/web` , `apps/workers` , `packages/country-packs`) and fails if a workspace with a `package.json` isn't mapped or excluded — see §5.
`db:migrate`	`node scripts/db/migrate.js`	Applies `db/schema.sql` (see §4).

Script	Command	Purpose
`db:seed:dev-user`	`node scripts/db/seed-dev-user.js`	Creates/updates one local login user. Dev convenience only.

`apps/web/package.json`'s own scripts (what actually runs the Next.js app):

Script	Command
`dev`	`next dev`
`build`	`next build`
`start`	`next start`
`test`	`jest`

There is **no separate database migration step baked into the build or start scripts** — `pnpm build` / `pnpm start` do not touch the database at all. Applying `db/schema.sql` is a manual, deliberate step you run yourself (§4). This is a documented, known gap in the project's own history (see `apps/web/src/lib/db/schema-check.ts`'s comment: "there is no automated migration-on-deploy step, so this is a manual, easy-to-miss action") — which is exactly why a startup schema check exists (§2.3) to catch it loudly rather than silently.

2.3 What happens at process start

`apps/web/instrumentation.ts` registers Next.js's standard `register()` startup hook. When the Node.js runtime boots (this does not run under the Edge runtime), and only when `NODE_ENV=production`, it:

1. Calls `validateStartupEnv()` — throws and **refuses to boot** if `DATABASE_URL` is missing, or if `JWT_SECRET` / `WEBHOOK_SIGNING_SECRET` / `ADMIN_ACCESS_TOKEN` / `ADMIN_SESSION_SECRET` / `CRON_SECRET` / `REDIS_URL` / any of the four `S3_*` core variables are missing **or** still set to the codebase's known hardcoded dev placeholder values.
2. Calls `validateStartupSchema()` — connects to `DATABASE_URL` and checks that a fixed list of tables the current build's code depends on actually exist (today: `DocumentDraft`). If a required table is missing, the process refuses to boot with an explicit instruction to run `scripts/db/migrate.js`.

In any non-production `NODE_ENV` (local dev, test, staging without that flag set), both checks are **no-ops** — the app boots regardless, using the insecure fallback secrets. This means: **setting `NODE_ENV=production` is what turns on the safety net.** A production deployment that runs with `NODE_ENV` unset or `=development` will start successfully on insecure defaults and an unmigrated schema — silently, with no error.

2.4 Cron

`GET /api/cron/seven-day-preparation` is meant to be invoked once daily by an external scheduler (the code comments reference a Vercel Cron Job), never by a browser. It is authenticated by a `Bearer <CRON_SECRET>` header compared in constant time. Whatever scheduling mechanism your deployment platform provides, point it at this route with that header.

---

3. Configuration

All configuration is environment variables, read directly via `process.env.*` — there is no config file, no secrets manager integration, and no `.env` file loaded automatically in production (Next.js only auto-loads `.env.local` etc. in the same way it always does; in a real deployment you set these in your hosting platform's environment configuration). `.env.example` at the repo root is the canonical, maintained reference; the tables below were verified against it and against every `process.env.` read in `apps/web/src`.

A general Next.js rule that applies here too: any variable prefixed `NEXT_PUBLIC_` is inlined into the client-side JavaScript bundle at build time and is not a secret — only `NEXT_PUBLIC_APP_URL` and `NEXT_PUBLIC_SIMULATED_TENANT_ID` carry that prefix in this codebase.

3.1 Database

Variable	Configures	If missing
`DATABASE_URL`	The app's runtime PostgreSQL connection (`apps/web/src/lib/db/db-client.ts`). **Must** be the least-privilege `nextcase_app` role created by `db/schema.sql` — never a superuser/owner role, which would silently bypass RLS.	Any DB-touching request throws `DATABASE_URL is not set...` (a clean, immediate error, not a crash of the whole process). In production, `instrumentation.ts` refuses to boot at all.
`MIGRATION_DATABASE_URL`	A separate admin/DDL connection used **only** by `scripts/db/migrate.js` , which needs `CREATE TABLE/ROLE/POLICY` privileges. Falls back to `DATABASE_URL` if unset.	`scripts/db/migrate.js` exits immediately with an explicit error if neither is set.

3.2 Redis (rate limiting / caching)

Variable	Configures	If missing
`REDIS_URL`	`ioredis` connection (`apps/web/src/lib/security/redis-client.ts`), used to share login and admin-login rate-limit counters across instances (`redis-rate-limit.ts`), and by the AI context cache.	`getRedisClient()` returns `null` rather than throwing; every caller falls back to a per-process, in-memory rate limiter. Functionally the app keeps working in a single-instance deployment; behind a load balancer with multiple instances, each instance enforces its own separate limit, which is not a real limit. If Redis is configured but becomes unreachable mid-request, the rate limiter fails open to the in-memory fallback rather than blocking the request. Required in production (startup check).

3.3 Object storage (S3-compatible)

Variable	Configures	If missing
`S3_ENDPOINT`	Base URL of the S3-API-compatible service (`apps/web/src/lib/storage/object-storage.ts`).	Together with the three below: if any is unset, `isObjectStorageConfigured()` is `false` and `putObject` / `getObject` / `deleteObject` throw `OBJECT_STORAGE_NOT_CONFIGURED`

Variable	Configures	If missing
		— the upload/download/preview routes surface this as a clear, defined error rather than a generic crash. Required in production.
`S3_BUCKET`	Target bucket name.	See above.
`S3_ACCESS_KEY_ID`	Access key credential.	See above.
`S3_SECRET_ACCESS_KEY`	Secret key credential.	See above.
`S3_REGION`	AWS region field (required by the S3 API shape even for providers that ignore it).	Defaults to `us-east-1`.
`S3_FORCE_PATH_STYLE`	Path-style (`http://host/bucket/key`) vs. virtual-hosted-style addressing.	Defaults to `true` (path-style) unless explicitly set to the string `"false"` — path-style is what MinIO, s3rver, and most self-hosted S3-compatible services expect.

3.4 Security / session secrets

Every secret in this group follows the **same pattern**: it has a hardcoded, publicly-visible-in-source fallback value used for local dev ergonomics, and `apps/web/src/lib/security/env-validation.ts` refuses to let the process boot in `NODE_ENV=production` if the variable is either unset or still equal to that known placeholder.

Variable	Configures	If missing (non-production)	In production
`JWT_SECRET`	Signs/verifies regular user session JWTs (`lib/auth/jwt.ts`, `proxy.ts`).	Falls back to a hardcoded placeholder string — sessions still work, but are forgeable by anyone who reads the source.	Boot refused if unset or equal to the placeholder.
`ADMIN_SESSION_SECRET`	Signs/verifies the admin console's own session JWT, deliberately separate from `JWT_SECRET` so the two trust boundaries don't share a key.	Same placeholder-fallback behavior.	Boot refused if unset/placeholder.
`ADMIN_ACCESS_TOKEN`	The actual admin login secret checked (constant-time comparison) by `POST /api/admin/session`.	Falls back to a hardcoded placeholder.	Boot refused if unset/placeholder.
`WEBHOOK_SIGNING_SECRET`	HMAC-SHA256 signing/verification for inbound `POST /api/webhooks` requests (`lib/security/webhook-signature.ts`). Senders must sign `${timestamp}.${rawBody}` and send `sha256=<hex>` in `x-nextcase-signature` plus a fresh `x-nextcase-timestamp` (5-minute tolerance).	Falls back to a hardcoded placeholder.	Boot refused if unset/placeholder.

Variable	Configures	If missing (non-production)	In production
`CRON_SECRET`	Bearer-token credential required by `GET /api/cron/seven-day-preparation` — the only thing protecting that route.	Falls back to a hardcoded placeholder.	Boot refused if unset/placeholder.
`ALLOWED_ORIGINS`	Comma-separated allowlist for Origin/Referer-based CSRF verification on state-changing routes (login, logout, upload) — `lib/security/origin-check.ts`.	Defaults to `http://localhost:3000`. Not enforced by the production startup check, but must be set to your real app URL(s) in any shared environment or those routes will reject every real browser request.	Not a hard boot gate — set it anyway.

3.5 AI providers (OpenAI / Anthropic)

Provider-agnostic: nothing outside `lib/ai/providers/*` imports a vendor SDK directly.

Variable	Configures	If missing
`AI_PROVIDER`	`"openai"` (default) or `"anthropic"` — selects which vendor backs `POST /api/ai/ask` (RAG chat) and `POST /api/ai/draft`.	Any other value throws `AIProviderNotConfiguredError` at resolution time.
`OPENAI_API_KEY`	API key when `AI_PROVIDER=openai` (the default).	If unset, `getLLMProvider()` throws a clear `AIProviderNotConfiguredError` naming the missing key — routes surface this as a defined "not configured" response (documented in the project's own build history as a 503) rather than crashing.
`OPENAI_MODEL`	Model name.	Defaults to `gpt-4o-mini`.
`ANTHROPIC_API_KEY`	API key when `AI_PROVIDER=anthropic`.	Same "not configured" behavior as above.
`ANTHROPIC_MODEL`	Model name.	Defaults to `claude-3-5-haiku-latest`.

`POST /api/ai/ask` (RAG chat) additionally returns a defined "no context found" result — without ever calling the LLM — when nothing relevant is indexed for the tenant, independent of whether an AI provider key is configured at all.

3.6 Embeddings (document search/indexing)

Variable	Configures	If missing
`EMBEDDING_API_BASE_URL`	Base URL of any HTTP endpoint implementing the OpenAI embeddings API shape (`POST {base}/embeddings`) — OpenAI itself, or self-hosted/gateway options (vLLM,	If either this or the key below is unset, `getEmbeddingProvider()` silently falls back to a deterministic **local "feature hashing"** embedding — a real, testable technique (shared vocabulary lands in similar output dimensions) but explicitly **not semantically meaningful** the way a trained model's

Variable	Configures	If missing
	Ollama's OpenAI-compatible mode, LiteLLM).	output is. Indexing and hybrid search remain fully functional in this mode; similarity-search *quality* is the only thing that degrades. No error is raised either way.
`EMBEDDING_API_KEY`	API key for the endpoint above.	See above.
`EMBEDDING_MODEL`	Model name passed to the endpoint.	Defaults to `text-embedding-3-small`.

3.7 Judgment Research

Variable	Configures	If missing
`JUDGMENT_PROVIDER`	Selects which registered judgment-research provider backs `GET /api/judgments/search` (`lib/judgments/config.ts`).	Defaults to the literal id `"placeholder"`. As of this build, only a placeholder provider is registered — **no real external legal-research vendor is connected**, regardless of this variable's value. This is architecture, not a missing-config bug; see the Facts Sheet.

3.8 Email (Resend) and SMS (Twilio)

Both libraries exist at the abstraction/provider level and are fully implemented with real vendor SDKs, retries, and error handling — but **no route in the application currently calls either one**. Hearing-reminder delivery today is in-app notifications only (`Notification` table, bell icon), not email or SMS. Configure these only in anticipation of a future feature that wires them in; they have zero effect on current behavior.

Variable	Configures	If missing
`EMAIL_PROVIDER`	Only `"resend"` is a supported value today.	Any other value throws `EmailProviderNotConfiguredError`.
`RESEND_API_KEY`	Resend API key.	If unset (with `EMAIL_FROM_ADDRESS`), `getEmailProvider()` throws a clear "not configured" error — moot today since nothing calls it.
`EMAIL_FROM_ADDRESS`	Verified sending address.	Same as above.
`SMS_PROVIDER`	Only `"twilio"` is supported today.	Any other value throws `SmsProviderNotConfiguredError`.
`TWILIO_ACCOUNT_SID`	Twilio account SID.	If unset (with the two below), throws a clear "not configured" error.
`TWILIO_AUTH_TOKEN`	Twilio auth token.	See above.
`TWILIO_FROM_NUMBER`	Sending phone number.	See above.

3.9 Billing (Stripe)

Variable	Configures	If missing
`PAYMENT_PROVIDER`	Only `"stripe"` is supported today.	Any other value throws `PaymentProviderNotConfiguredError`.
`STRIPE_SECRET_KEY`	Stripe secret API key, used by `POST /api/billing/checkout` to create a real Checkout Session for an AI Credits wallet top-up.	If unset (with the key below), `getPaymentProvider()` throws a clear "not configured" error.
`STRIPE_WEBHOOK_SECRET`	Verifies `POST /api/billing/webhook`'s inbound Stripe signature before crediting a tenant's wallet.	Same as above — an unverifiable webhook is never trusted.

Per the Facts Sheet: this credits a `TenantWallet.balance` row via a real Stripe Checkout Session and a real, signature-verified webhook — but AI Credits balances/plans are otherwise a local/mock persistence layer, not a full commercial billing ledger.

3.10 Application / miscellaneous

Variable	Configures	If missing
`NEXT_PUBLIC_APP_URL`	Public base URL used for Stripe Checkout success/cancel redirect targets, the OpenGraph `metadataBase`, `robots.ts`, and `sitemap.ts`.	Defaults to `http://localhost:3000` — fine for local dev, wrong for any real deployment (redirects and sitemap URLs will point at localhost).
`PRODUCT_REVIEW_MODE`	Opt-in, secure-by-default feature flag (`lib/beta/demo-data.ts`, `proxy.ts`). When `"true"`, additionally exposes the Ask AI Action Card, AI Credits & Usage page, and one fixed synthetic demo Matter Workspace to **unauthenticated** visitors, using only static sample data — never a database query, never real tenant data. All write routes and every other real-data GET are completely unaffected.	Defaults to off (any value other than the exact string `"true"`, including unset).
`NODE_ENV`	Standard Node environment flag. Gates the two production-only startup checks (§2.3), the `Secure` cookie attribute on session/admin cookies, and CI's own settings.	Not setting it to `production` in a real deployment silently disables the startup safety net — see §2.3.
`BETA_PREVIEW_ENABLED`	**Retired — has no effect at all.** Formerly gated the same preview surface `PRODUCT_REVIEW_MODE` now controls. A regression test in the codebase explicitly asserts that setting this to `"true"`	N/A

Variable	Configures	If missing
	changes nothing. Do not rely on it; it is documented here only so an operator who finds it in an old runbook knows it's dead.	
`NEXT_PUBLIC_SIMULATED_TENANT_ID`	Read by exactly one module, `lib/tenant-context.ts` (`getActiveTenantId`) — a UUID-format tenant id it throws `SECURE_ACCESS_DENIED` without. Nothing else in the application currently imports this function; every real API route resolves tenant identity from the verified session cookie instead (see §4.2). Treat this as dead/legacy code from an earlier development phase, not a live security control.	N/A for current routes.

3.11 Development-only (seed script)

Variable	Configures	If missing
`DEV_SEED_USER_PASSWORD`	Password for the local dev login user created by `pnpm db:seed:dev-user` .	Script refuses to run at all — deliberately, so no default password is ever committed.
`DEV_SEED_USER_EMAIL`	Login email for that user.	Defaults to `dev@nextcase.local` .
`DEV_SEED_TENANT_ID`	Tenant the seeded user belongs to.	Defaults to the fixed UUID `00000000-0000-0000-0000-000000000001` .

---

4. Database

4.1 Schema management

`db/schema.sql` is the single, authoritative, **hand-written and idempotent** schema file — there is no ORM and no numbered migration framework. Every statement is written to be safely re-run against an already-migrated database (`CREATE TABLE IF NOT EXISTS` , `CREATE INDEX IF NOT EXISTS` , `DROP POLICY IF EXISTS` before `CREATE POLICY` , guarded `DO $$ ... $$` blocks for constraints). Applying it is:

```
MIGRATION_DATABASE_URL=postgres://<admin-role>:<password>@<host>:5432/<db> \
  node scripts/db/migrate.js
```

This connection **must** have DDL privileges (it creates tables, the `nextcase_app` role, indexes, and RLS policies) — an admin/superuser connection, e.g. Supabase's default `postgres` role, is the appropriate choice here specifically, and only here. Run this:

- On first provisioning of any environment's database.
- After pulling any update that touched `db/schema.sql` (see §5) — there is no automated migration-on-deploy step; you must run this yourself.

After the schema is applied, set a real password on the application role out-of-band (the schema creates the role with `LOGIN NOSUPERUSER NOCREATEDB NOCREATEROLE NOBYPASSRLS` but no password, since a password can't live in a file committed to git):

```
ALTER ROLE nextcase_app WITH PASSWORD '<a-real-password>';
```

Then point `DATABASE_URL` at that role, never at the admin/migration role.

4.2 Row-Level Security / tenant isolation — the real model

This is the core security boundary of the entire application and must **never be disabled** in any environment that holds more than one tenant's data. Concretely, from `db/schema.sql`:

1. Every domain table has a `tenant_id UUID` column referencing `"Tenant"`.
2. `ALTER TABLE "<table>" ENABLE ROW LEVEL SECURITY;` **and** `ALTER TABLE "<table>" FORCE ROW LEVEL SECURITY;` are both run for every one of the ~25 RLS-protected tables. `FORCE` is not redundant: without it, PostgreSQL exempts the table's **owner** from its own RLS policies, and the schema's own comment notes the application's runtime role is expected to be the table owner in a typical single-role deployment — so `ENABLE` alone would make RLS a silent no-op for all real traffic while still displaying as "enabled".
3. A single reusable policy, `tenant_isolation_policy`, is applied per table: `USING ("tenant_id" = get_active_session_tenant())`.
4. `get_active_session_tenant()` reads the session-scoped setting `nextcase.current_tenant_id` (via `current_setting(...)`) and **raises an exception** (`SECURE_ACCESS_DENIED`) if it is unset or empty — there is no default-open fallback at the database level.
5. The application binds that setting for the lifetime of one transaction only, via `SELECT set_config('nextcase.current_tenant_id', $1, true)` inside `DatabaseClient.execute(tenantId, ...)` — every real query goes through this path. A small, explicitly-named exception, `DatabaseClient.executeSystem(...)`, runs with no tenant context bound at all, and is only used against the two tables that have no RLS at all by design (`"Tenant"`, and login's tenant-agnostic user lookup by email).
6. The `nextcase_app` role is granted plain `SELECT, INSERT, UPDATE, DELETE` — no DDL, no `BYPASSRLS`. A handful of **append-only audit/ledger tables** (`AiUsageEvent`, `DocumentAccessEvent`, `CourtNote`, `MatterPreparationReminder`, `MatterClosureRecord`, `MatterReopeningRecord`, `MatterAuditEvent`, `WalletTransactionRecord`) additionally have `UPDATE, DELETE` **revoked** from that role — a real grant restriction enforced by PostgreSQL, not just an application convention, so even a bug in application code cannot silently rewrite a hearing record or a financial transaction after the fact.
7. Foreign keys are **never** trusted as a tenant boundary by themselves — the schema's own comments note (learned from a real, fixed bug in `DocumentEnvelope.case_id`) that PostgreSQL FK constraint checks always bypass RLS. Every table that matters for isolation carries its own `tenant_id` + its own policy, even where a parent-table FK already exists.

Operational implication: never connect the application as a superuser or as a table-owning/admin role. Never grant `BYPASSRLS` to `nextcase_app`. Never run application traffic through the `MIGRATION_DATABASE_URL` connection. If you are on Supabase, do not point `DATABASE_URL` at the project's default `postgres` role — it bypasses RLS unconditionally, which would silently defeat every tenant boundary described above while looking like normal operation.

4.3 Backups

There is nothing NextCaseHQ-specific about backup mechanics — it is a standard PostgreSQL database, and standard PostgreSQL practice applies:

- **Logical backups**: `pg_dump --format=custom` (or plain SQL) on a schedule, retained per your compliance requirements. Given this is a litigation records system, treat retention and access to backup files with at least the same care as the live database — backups contain the same tenant-isolated client/case data with none of the RLS protection once restored to a plain file.
- **Point-in-time recovery**: if your PostgreSQL provider supports WAL archiving/PITR (most managed providers do), enable it — this system's append-only audit tables (Court Notes, AI usage events, wallet transactions) are specifically designed so history is never silently rewritten; a good backup/PITR strategy is what makes that guarantee actually recoverable after an incident, not just a schema-level promise.
- **Test your restores.** Restoring `db/schema.sql`'s DDL (roles, RLS policies, the `vector` / `pg_trgm` extensions) must succeed against the same PostgreSQL major version and available extensions as production — a restore target without `pgvector` installed will fail at `CREATE EXTENSION IF NOT EXISTS vector`.
- Never restore a backup by replaying it through the application's `nextcase_app` connection — restore through an admin connection, exactly like the migration step, then verify RLS is still `FORCE`d (a `pg_dump` / `pg_restore` round-trip preserves this, but confirm it after any manual schema surgery).

---

5. Updates

5.1 The project's actual validation ritual

This codebase's own history (`docs/PENDING_INTEGRATION_REGISTER.md`) consistently describes every merged change as validated by the same sequence before being considered safe to ship: **full automated test suite** → **`pnpm typecheck`** → **`pnpm build`**, plus, for anything touching a live route, manual verification against a real running server (not mocks) — e.g. "Verified: 33 new tests... full 242-test suite passing... real production-server verification confirming the exact boundary." Follow the same gate when you pull and deploy an update:

```
git pull
pnpm install --frozen-lockfile
pnpm typecheck        # node scripts/typescript_validation.js — tsc --noEmit per workspace
pnpm --filter web test    # jest — apps/web's real unit/integration suite
pnpm build            # turbo build
```

This mirrors what CI actually runs. `.github/workflows/ci.yml` has four jobs (`lint`, `typecheck`, `build`, `test`) that all run on every push to `main` and every pull request; `.github/workflows/compiler-sentinel.yml` runs lint → typecheck → build again (with a "CERTIFIED FOR MERGE" / "COMPILER VALIDATION FAILED" banner) on pushes to `main`, `develop`, and `feat/**` / `fix/**` branches. The `test` job specifically spins up real service containers — `pgvector/pgvector:pg16` for Postgres and `redis:7` for Redis — applies `db/schema.sql` via `pnpm db:migrate`, provisions the least-privilege `nextcase_app` role, and starts a real `s3rver` instance before running the Jest suite — i.e. the automated tests exercise real RLS, real Redis, and real S3-API semantics, not mocks of them, for the parts of the stack that can be run in CI this way.

5.2 Deploy steps

1. Pull the new code.
2. Run the validation ritual above locally or in CI before touching production.
3. If the change touched `db/schema.sql`, apply it against the target environment's database **before** deploying the new application code (the additive, `IF NOT EXISTS`-guarded style means this is safe to run ahead of the code that depends on it):

`MIGRATION_DATABASE_URL=<admin-connection> node scripts/db/migrate.js`

4. Deploy the new `apps/web` build and restart/replace the running process(es). Because `instrumentation.ts` (§2.3) checks for known required tables at boot in production, a deploy that shipped code depending on a table you forgot to migrate will refuse to start with an explicit error naming the missing table — rather than serving a confusing per-request 500 once real traffic hits the new code path.
5. Confirm `/api/health` responds (§6) and spot-check a real login and one real read/write flow (e.g. open `/matters`, view a Matter).

5.3 Rollback

Since migrations are additive (new nullable columns, new tables, new constraints that don't invalidate existing rows), rolling back the **application code** to a previous build while leaving a newer schema applied is generally safe — older code simply doesn't read the new columns/tables. Rolling back the **schema** itself is not something `db/schema.sql` is designed to do (there are no corresponding "down" migrations); if you must undo a schema change, do it by hand, deliberately, against a verified-good backup rather than by scripted rollback.

---

6. Monitoring

NextCaseHQ ships three HTTP endpoints under `/api/*/health`, and their real behavior is worth understanding precisely — it is not what the names alone suggest.

6.1 `GET /api/health`

Reads `apps/web/src/app/api/health/route.ts`: this endpoint returns a **fixed, hardcoded JSON payload** — `system_health: "STABLE"`, `built_to_date`, `current_target`, `backlog_items`, a live timestamp, and an `edge_runtime` flag — plus a latency header showing how long it took to build that same hardcoded object. **It does not check the database, Redis, object storage, or any AI provider.** A `200` from this endpoint tells you only that the Next.js process is running and able to serve a request; it tells you nothing about the health of any dependency. Use it as a bare liveness probe (is the process up and responding at all), never as a readiness or dependency-health check.

6.2 `GET /api/admin/health`

Reads `apps/web/src/app/api/admin/health/route.ts`: this endpoint is likewise **fully hardcoded** — `status: "STABLE"`, and a `metrics` object with fixed placeholder values (`activeTenants: 142`, `latencyMs: 1.84`, `systemScore: "100%"`). These numbers do not come from the database or any real metric source; they are static sample values baked into the route. Do not build alerting or a dashboard on this endpoint's numeric fields — they never change and do not reflect real tenant counts, latency, or system health. It shares the same limitation as `/api/health`: it confirms the process can respond, nothing more.

6.3 `GET /api/matters/[id]/health`

This is a different concept entirely — not a system/infrastructure health check, but the product's **Matter Health** feature: a live-derived summary of one specific Matter (current stage, last hearing date/forum/note, next hearing date, pending action count, a needs-attention flag), computed from real database rows for that Matter. It is a product feature for advocates, not an operator-facing monitoring endpoint — do not point uptime monitoring at it, and do not confuse it with §6.1/§6.2 above.

6.4 What to actually monitor, given the above

Since neither operator-facing health endpoint checks real dependencies today, build your actual monitoring around:

- **Process-level liveness**: `/api/health` (or your platform's own process supervisor) for "is the app up."
- **Database**: monitor PostgreSQL directly (connection count, replication lag if applicable, `pg_stat_activity`) through your database provider's own tooling — not through the application.
- **Redis**: monitor directly through your Redis provider — the app fails open (in-memory fallback) silently on Redis errors (only logged to stderr as `[REDIS] connection error: ...` / `[REDIS] rate limit check failed, falling back to in-memory: ...`), so an operator watching only HTTP status codes would never see a Redis outage reflected there.
- **Object storage**: monitor your S3-compatible provider directly; a misconfigured/unreachable bucket surfaces to end users as upload/download failures, not as a change in either health endpoint.
- **Application logs**: the startup checks in §2.3 log/throw with clear, greppable messages (`Startup environment validation failed:`, `Startup schema validation failed:`) — capture process stdout/stderr so a failed boot due to a missing secret or unmigrated table is visible, since in `NODE_ENV=production` that failure prevents the process from ever reaching a state where an HTTP health check could even be hit.

---

7. Troubleshooting

The scenarios below are all backed directly by code paths described in §3 and §6 — not hypothetical.

Document upload/download/preview fails with `OBJECT_STORAGE_NOT_CONFIGURED` or a related 5xx. One or more of `S3_ENDPOINT` / `S3_BUCKET` / `S3_ACCESS_KEY_ID` / `S3_SECRET_ACCESS_KEY` is unset or wrong. Confirm all four are set and that the endpoint is reachable from the running app instance (`object-storage.ts` requires all four together — there's no partial-credential fallback).

`POST /api/ai/ask` or `POST /api/ai/draft` returns a "not configured" / 5xx AI provider error. Check `AI_PROVIDER` (defaults to `openai` if unset) and confirm the matching key is set: `OPENAI_API_KEY` for `openai`, `ANTHROPIC_API_KEY` for `anthropic`. Setting `AI_PROVIDER` to anything other than those two literal values throws immediately. This is expected, documented "not configured" behavior, not a bug — the project's own history records verifying this exact boundary (503 with an indexed context but no vendor key set).

Semantic search results feel low-quality / unrelated documents rank highly. If `EMBEDDING_API_BASE_URL` / `EMBEDDING_API_KEY` are unset, the app is silently using the local deterministic "feature hashing" embedding fallback — functional, real, but explicitly not semantically meaningful. This produces working-but-mediocre search, not an error. Set both variables (and optionally `EMBEDDING_MODEL`) to get real vector similarity quality.

Login or admin-login rate limiting seems to reset per-instance / users report inconsistent lockouts behind a load balancer. `REDIS_URL` is unset (or unreachable), so each app instance is enforcing its own separate in-memory counter — expected behavior per §3.2, not a bug, but not safe for a multi-instance deployment. Provision Redis and set `REDIS_URL`.

A fresh production deployment refuses to start, logging `Startup environment validation failed: ...`. Working as intended (§2.3) — `NODE_ENV=production` plus a missing or still-placeholder value for `JWT_SECRET`, `WEBHOOK_SIGNING_SECRET`, `ADMIN_ACCESS_TOKEN`, `ADMIN_SESSION_SECRET`, `CRON_SECRET`, `REDIS_URL`, or any `S3_*` core variable. Set real values for whichever variable the error names.

A fresh production deployment refuses to start, logging `Startup schema validation failed: ... missing required table(s): ...`. The application code was deployed ahead of a database migration. Run

`MIGRATION_DATABASE_URL=<admin-connection> node scripts/db/migrate.js` against that environment's database, then restart the process.

A deployment with insecure default secrets appears to "work fine." If `NODE_ENV` is not exactly `production`, none of the startup safety checks in §2.3 run at all — the app boots happily on the hardcoded placeholder secrets embedded in the source. This is silent by design in non-production environments. Always explicitly set `NODE_ENV=production` for any shared/production deployment, and treat "the app started" as meaningless evidence of correct configuration unless that variable is set.

Setting `BETA_PREVIEW_ENABLED=true` doesn't unlock the review-mode preview surface. That variable is retired and deliberately ignored (a regression test asserts this). Use `PRODUCT_REVIEW_MODE=true` instead.

Hearing reminders never arrive by email or SMS. By design, per the Facts Sheet — the Resend/Twilio abstractions exist and are fully implemented at the library level, but no route in this build calls either. Reminders are delivered only as in-app notifications today. Configuring `RESEND_API_KEY` / `TWILIO_*` has no visible effect until a future feature wires them into a real call site.

---

8. Maintenance

Log review. Watch process stdout/stderr for the greppable markers this codebase already uses consistently: `[REDIS]` (connection errors, rate-limit fallbacks), `[db:migrate]` (migration run output), `[seed-dev-user]`, `[SEVEN_DAY_PREPARATION_CRON] run failed:` (the daily reminder job's own error logging), and the startup-validation error blocks from §2.3/§7. None of these are shipped to an external log aggregator by the application itself — wire your platform's log capture to stdout/stderr.

Dependency updates. `pnpm install --frozen-lockfile` in CI and deployment means dependency versions only change when `pnpm-lock.yaml` is deliberately updated (e.g. `pnpm update` run locally, reviewed, and committed) — there is no auto-update mechanism. Before bumping anything in `apps/web/package.json` (particularly `@aws-sdk/client-s3`, `pg`, `ioredis`, `openai`, `@anthropic-ai/sdk`, `stripe`, `resend`, `twilio` — the vendor SDKs every provider abstraction wraps), run the full validation ritual in §5.1 before deploying.

Rotating secrets. For each of `JWT_SECRET`, `ADMIN_SESSION_SECRET`, `ADMIN_ACCESS_TOKEN`, `WEBHOOK_SIGNING_SECRET`, and `CRON_SECRET`: generate a new value (e.g. `openssl rand -base64 32`), set it in the deployment environment, and restart the app. Be aware of the real operational effect of each:

- Rotating `JWT_SECRET` or `ADMIN_SESSION_SECRET` invalidates every currently-issued session/admin-session cookie immediately — all users (or all admin operators) will need to log in again.
- Rotating `WEBHOOK_SIGNING_SECRET` requires coordinating with whatever external system sends signed requests to `POST /api/webhooks` — it must start signing with the new secret at the same time, or its requests will fail signature verification.
- Rotating `CRON_SECRET` requires updating whatever external scheduler calls `GET /api/cron/seven-day-preparation` with the new bearer token at the same time, or the daily reminder job will start failing with 401s.
- Rotating `ADMIN_ACCESS_TOKEN` requires distributing the new value to whoever is authorized to sign into the admin console — it is a single shared operator secret today, not per-admin individual credentials (a known, deliberately recorded limitation, not an oversight).

Set a recurring reminder (quarterly is a reasonable default for a system handling client/litigation data) rather than relying on memory.

Database housekeeping. The append-only audit/ledger tables (`CourtNote` , `AiUsageEvent` , `DocumentAccessEvent` , `MatterPreparationReminder` , `MatterClosureRecord` , `MatterReopeningRecord` , `MatterAuditEvent` , `WalletTransactionRecord`) only grow — there is no automated pruning, and `UPDATE` / `DELETE` are revoked from the application role by design, so routine "cleanup" of these tables is not something the app can do to itself. If retention limits are ever required, that is a deliberate, separate administrative decision (direct admin-role SQL against a backup- verified environment), not a routine task to script casually.

Re-running the schema. `node scripts/db/migrate.js` is explicitly safe to re-run at any time against an already-migrated database — every statement in `db/schema.sql` is written to be idempotent. Re-running it periodically (e.g. after any doubt about whether a hotfix's schema change was actually applied to a given environment) is a reasonable, low-risk sanity check, not something to avoid.

This manual was written by inspecting `db/schema.sql` , `scripts/db/migrate.js` , `scripts/db/seed-dev-user.js` , `package.json` , `apps/web/package.json` , `.env.example` , `apps/web/instrumentation.ts` , `apps/web/src/lib/security/env-validation.ts` , `apps/web/src/lib/db/schema-check.ts` , the provider-abstraction modules under `apps/web/src/lib/{ai,billing,messaging,search,storage,judgments}` , the two `/api/*/health` route handlers, `.github/workflows/ci.yml` , `.github/workflows/compiler-sentinel.yml` , and `docs/PENDING_INTEGRATION_REGISTER.md` , alongside `docs/knowledge-base/FACTS_SHEET.md` for product-level context.